Principia Novae Mathematicae

Filippo Mario Oppo

2026

Contents

Prefazione **5**

1 Fondamenti **7**

2 Addizione **9**

3 Sottrazione **11**

4 Moltiplicazione **13**

5 Divisione **15**

6 Tabelle ed Esempi **17**
6.1 Tabella di conversione rapida decimale $\leftrightarrow$ Novae . . . 17
6.2 Tabelle delle operazioni fondamentali . . . 17
6.2.1 Tabella di addizione . . . 17
6.2.2 Tabella di sottrazione . . . 18
6.2.3 Tabella di moltiplicazione . . . 18
6.3 Esempi di calcolo . . . 18
6.3.1 Addizione: $5 + 7 = 03$ . . . 18
6.3.2 Sottrazione: $8 - 2 = 5$ . . . 19
6.3.3 Moltiplicazione: $9 \times 0 = 9$ . . . 19
6.3.4 Divisione: $8 \div 2 = 2$ . . . 19
6.3.5 Espressione concatenata: $89 \times 0 + 1 = 91$ . . . 19
6.3.6 Esempio con segni: $-43 - 3 = -47$ . . . 19
6.3.7 Zeri posizionali: $101 \rightarrow 90$ . . . 19

7 Proprietà Algebriche **21**
7.1 Commutatività dell'addizione . . . 21
7.2 Commutatività della moltiplicazione . . . 21
7.3 Associatività dell'addizione . . . 21
7.4 Associatività della moltiplicazione . . . 21
7.5 Distributività . . . 22
7.6 Elemento neutro . . . 22
7.7 Opposto . . . 22

8 Conversione Decimale **23**
8.1 Da decimale a Novae . . . 23
8.2 Da Novae a decimale . . . 23

9 Frazioni **25**
9.1 Notazione con virgola . . . 25
9.2 Operazioni con frazioni . . . 25

10 Ordinamento **27**

11 Confronto con il Sistema Decimale **29**
11.1 Simmetria e assenza di zeri multipli 29
11.2 Rappresentazione compatta 29
11.3 Addizione e riporto 29

12 Ipotesi e Prospettive Fisiche **31**
12.1 Grafo icosaedrico e stati quantistici 31
12.2 Rappresentazione delle particelle 31
12.3 Reti neurali geometriche (FINNN) 31

13 Implementazione Hardware **33**
13.1 ALU per Novae 33
13.2 Prestazioni 33
13.3 Validazione empirica 33

14 Prospettive Future **35**

A Tabelle Complete **37**

B Codice Sorgente del Motore **39**

Prefazione

Il presente trattato getta le fondamenta di un nuovo sistema numerico e geometrico, detto **Novae**, nato dall'esigenza di descrivere una realtà discreta, finita e priva del vuoto assoluto. Ispirandosi ai *Philosophiæ Naturalis Principia Mathematica* di Newton, l'opera procede per definizioni, assiomi e proposizioni rigorosamente dimostrate.

I quattro Assiomi Zero – tridimensionalità, discretezza, inesistenza del vuoto e kissing number 12 – costituiscono il sostrato filosofico da cui scaturisce l'aritmetica dei 21 simboli. La notazione compatta, che usa le cifre nude $0, 1, \ldots, 9$ per i valori positivi e il segno $-$ per i negativi, permette di maneggiare i numeri con la stessa familiarità del sistema decimale, pur conservando una struttura algebrica più ricca e simmetrica.

Il Volume I è dedicato interamente all'aritmetica: addizione, sottrazione, moltiplicazione e divisione vengono definite in modo nativo, senza alcuna conversione decimale intermedia, e la loro correttezza è provata per via costruttiva. I volumi successivi tratteranno la geometria icosaedrica, la dinamica discreta e le applicazioni fisiche del framework Novae.

Chapter 1

Fondamenti

Definizione 1.1 (Simboli Novae)**.** *L'insieme ordinato dei 21 simboli fondamentali è*

$$\mathcal{S} = \{-9, -8, -7, -6, -5, -4, -3, -2, -1, -0,\ \varnothing,\ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}.$$

Il simbolo centrale $\varnothing$ è detto zero assoluto*. I simboli a sinistra di $\varnothing$ sono detti* antiparticelle *e si indicano con $-d$ (dove $d \in \{0, \ldots, 9\}$); il loro valore decimale è $-(d+1)$. I simboli a destra di $\varnothing$ sono detti* particelle *e si indicano con la cifra nuda d; il loro valore decimale è $d+1$. La funzione valore*

$$\sigma : \mathcal{S} \longrightarrow \{-10, -9, \ldots, 10\}$$

è definita da $\sigma(\varnothing) = 0$, $\sigma(-d) = -(d+1)$, $\sigma(d) = d+1$. Essa è biunivoca; la sua inversa si denota σ^{-1}.

Definizione 1.2 (Notazione compatta)**.** *Un numero Novae è una sequenza finita di simboli di $\mathcal{S}$, interpretata in base 10 con la consueta notazione posizionale. Per evitare la ripetizione dei segni si adotta la* notazione compatta*:*

- *Il numero zero si scrive $\varnothing$.*
- *Un numero positivo si scrive giustapponendo le sole parti numeriche dei simboli d (il segno $+$ è omesso). Ad esempio, la sequenza $0, 3$ (valori 1 e 4) si scrive 03 e rappresenta il valore $1 \cdot 10 + 4 = 14$.*
- *Un numero negativo si scrive anteponendo il segno $-$ e giustapponendo le parti numeriche dei simboli $-d$. Ad esempio, la sequenza $-1, -3$ (valori -2 e -4) si scrive -13 e rappresenta il valore $-(2 \cdot 10 + 4) = -24$.*

In questa notazione la cifra 0 rappresenta sempre il simbolo 0 (valore 1), mai lo zero assoluto.

Rappresentazione degli zeri posizionali. *Il simbolo $\varnothing$ (valore 0) può apparire all'interno della rappresentazione little-endian di un numero (ad esempio, per rappresentare il valore decimale 101, la lista little-endian è $[0, 9]$). Nella notazione compatta, $\varnothing$ viene convertito nella cifra 0 solo se si trova in posizioni intermedie o finali. Ad esempio, 101 si scrive 90 (decine 9, unità 0). Questo meccanismo garantisce che ogni numero intero abbia una rappresentazione compatta univoca.*

Assioma 1.1 (Non esistenza del nulla)**.** *Il simbolo $\varnothing$ non rappresenta uno stato di "vuoto" fisico, bensì una transizione o uno stato di equilibrio. Nella rappresentazione dei numeri esso denota il valore nullo, ma non esiste il nulla assoluto.*

Assioma 1.2 (Non esistenza dell'infinito)**.** *Non si fa uso dell'infinito attuale. Ogni grandezza è rappresentabile con un numero finito di simboli; tutte le operazioni sono definite su insiemi finiti e i risultati sono calcolabili in un numero finito di passi.*

Assioma 1.3 (Discretezza e simmetria)**.** *Lo spazio è modellato come un grafo i cui nodi sono sfere di raggio minimo* r_0*. In ogni punto il massimo numero di sfere tangenti è* 12 *(kissing number). La struttura locale è ovunque quella dell'icosaedro regolare (*13 *nodi). I* 21 *simboli corrispondono a stati di occupazione dei nodi.*

Chapter 2

Addizione

Definizione 2.1 (Addizione di simboli singoli (Nit))**.** *Siano $a, b \in \mathcal{S}$. L'addizione produce una coppia $(r, c) \in \mathcal{S} \times \{\varnothing, 0, -0\}$ definita dalla regola seguente. Posto $s = \sigma(a) + \sigma(b)$:*

- *se $-10 \leq s \leq 10$: $r = \sigma^{-1}(s)$, $c = \varnothing$;*
- *se $s > 10$: $r = \sigma^{-1}(s - 10)$, $c = 0$;*
- *se $s < -10$: $r = \sigma^{-1}(s + 10)$, $c = -0$.*

Il simbolo c è detto riporto *(carry).*

Lemma 2.1 (Riporto limitato)**.** *Per ogni $a, b \in \mathcal{S}$ il riporto c appartiene a $\{\varnothing, 0, -0\}$. Inoltre, l'addizione di due riporti $c_1, c_2 \in \{\varnothing, 0, -0\}$ produce un risultato in $\mathcal{S}$ e un riporto sempre $\varnothing$.*

Proof. La prima parte segue immediatamente dalla Definizione 2.1. Per la seconda si considerino tutte le $3 \times 3 = 9$ combinazioni possibili. In ogni caso il riporto finale è $\varnothing$. □

Definizione 2.2 (Addizione di numeri multi-cifra)**.** *Siano $A = a_k \dots a_0$ e $B = b_m \dots b_0$ due numeri Novae (little-endian: a_0, b_0 unità). L'addizione si esegue cifra per cifra, allineando le unità:*

1. *Sia $N = \max(k, m) + 1$. Si pongano $a_i = \varnothing$ per $i > k$ e $b_j = \varnothing$ per $j > m$.*
2. *Per $i = 0, \dots, N - 1$:*
   - *Calcolare $(s_1, c_1) = add_single(a_i, b_i)$.*
   - *Calcolare $(s_2, c_2) = add_single(s_1, carry)$, dove carry è il riporto proveniente dal passo precedente (inizialmente $\varnothing$).*
   - *Calcolare $(c_{next}, _) = add_single(c_1, c_2)$. Per il Lemma 2.1 il riporto di questa somma è $\varnothing$.*
   - *Porre la i-esima cifra del risultato uguale a s_2 e aggiornare $carry = c_{next}$.*
3. *Se al termine $carry \neq \varnothing$, aggiungerlo come cifra più significativa.*

Il risultato viene infine normalizzato rimuovendo eventuali $\varnothing$ finali (zeri non significativi).

Teorema 2.1 (Correttezza dell'addizione)**.** *L'algoritmo di addizione produce la rappresentazione canonica della somma decimale di A e B. Il riporto si propaga a cascata come nell'addizione decimale classica; la complessità temporale su architettura sequenziale è $O(n)$ rispetto al numero di cifre.*

Proof. Per induzione sulla lunghezza massima dei due addendi. Il passo base (una sola cifra) coincide con l'addizione singola. Il passo induttivo si basa sul fatto che l'addizione delle prime n cifre produce un risultato corretto e un riporto; l'aggiunta del riporto alla cifra successiva è un'addizione singola che, per il Lemma 2.1, non genera riporti maggiori di ± 0. La correttezza segue dall'associatività dell'addizione decimale e dalla biunivocità di σ. □

Chapter 3

Sottrazione

Definizione 3.1 (Sottrazione di simboli singoli)**.** *Siano $a, b \in \mathcal{S}$. La sottrazione produce una coppia $(r, b_{out}) \in \mathcal{S} \times \{\varnothing, 0, -0\}$ definita dalla regola seguente. Posto $d = \sigma(a) - \sigma(b)$:*

- *se $-10 \leq d \leq 10$: $r = \sigma^{-1}(d)$, $b_{out} = \varnothing$;*
- *se $d > 10$: $r = \sigma^{-1}(d - 10)$, $b_{out} = 0$;*
- *se $d < -10$: $r = \sigma^{-1}(d + 10)$, $b_{out} = -0$.*

Il simbolo b_{out} è detto borrow *(prestito).*

Lemma 3.1 (Borrow limitato)**.** *Per ogni $a, b \in \mathcal{S}$ il borrow b_{out} appartiene a $\{\varnothing, 0, -0\}$. Inoltre, l'addizione di due borrow b_1, b_2 (usando l'addizione singola) produce un risultato in $\mathcal{S}$ e un carry sempre $\varnothing$.*

Proof. Analoga al Lemma 2.1. □

Definizione 3.2 (Sottrazione di numeri multi-cifra)**.** *Siano A, B con $A \geq B$ (in valore assoluto). L'algoritmo little-endian è:*

1. *Allineare le cifre riempiendo con $\varnothing$ se necessario.*
2. *Inizializzare $borrow = \varnothing$.*
3. *Per $i = 0, \ldots, \max(len(A), len(B)) - 1$:*
   - $(s_1, b_1) = sub_single(a_i, b_i)$.
   - $(s_2, b_2) = sub_single(s_1, borrow)$.
   - $(b_{next}, _) = add_single(b_1, b_2)$ *(il riporto è $\varnothing$).*
   - $r_i = s_2$, $borrow = b_{next}$.
4. *Normalizzare rimuovendo $\varnothing$ finali.*

Se $A < B$ si scambiano i ruoli e si antepone il segno $-$ al risultato.

Teorema 3.1 (Correttezza della sottrazione)**.** *Se $A \geq B$, l'algoritmo produce la rappresentazione canonica di $A - B$. Il borrow si propaga a cascata analogamente al riporto dell'addizione.*

Proof. Induzione simile al Teorema 2.1, usando il Lemma 3.1. □

Chapter 4

Moltiplicazione

Definizione 4.1 (Moltiplicazione di simboli singoli)**.** *Siano $a, b \in \mathcal{S}$. La moltiplicazione produce una coppia $(u, t) \in \mathcal{S} \times \mathcal{S}$ dove u è la cifra delle unità e t quella delle decine. Posto $p = \sigma(a) \cdot \sigma(b)$,*

$$u = \begin{cases} \sigma^{-1}(p \bmod 10) & p \geq 0 \\ \sigma^{-1}(-(|p| \bmod 10)) & p < 0 \end{cases}$$

$$t = \begin{cases} \varnothing & \lfloor p/10 \rfloor = 0 \\ \sigma^{-1}(\lfloor p/10 \rfloor) & p \geq 0,\ \lfloor p/10 \rfloor \neq 0 \\ \sigma^{-1}(-|\lfloor p/10 \rfloor|) & p < 0,\ \lfloor p/10 \rfloor \neq 0 \end{cases}$$

con la convenzione $\sigma^{-1}(0) = \varnothing$.

Definizione 4.2 (Moltiplicazione di numeri multi-cifra)**.** *Siano A, B in notazione little-endian. L'algoritmo di moltiplicazione lunga è:*

1. *Inizializzare $P = [\varnothing]$ (zero).*
2. *Per ogni cifra a_i di A ($i = 0, \dots$):*
   - *Creare una riga R contenente i simboli $\varnothing$.*
   - *$carry = \varnothing$.*
   - *Per ogni cifra b_j di B ($j = 0, \dots$):*
     - *$(u, t) = mul_single(a_i, b_j)$.*
     - *$(s_1, c_1) = add_single(u, carry)$.*
     - *Aggiungere s_1 a R.*
     - *$(carry, _) = add_single(t, c_1)$.*
   - *Se $carry \neq \varnothing$, aggiungerlo a R.*
   - *$P = add_lists(P, R)$ (addizione nativa).*
3. *Normalizzare P.*

Teorema 4.1 (Correttezza della moltiplicazione)**.** *L'algoritmo produce la rappresentazione canonica del prodotto decimale $A \cdot B$.*

Proof. Conseguenza diretta dell'identità algebrica

$$\Big(\sum_i \sigma(a_i) 10^i\Big)\Big(\sum_j \sigma(b_j) 10^j\Big) = \sum_i \Big(\sum_j \sigma(a_i)\sigma(b_j) 10^{i+j}\Big)$$

e della correttezza dell'addizione (Teorema 2.1). □

Chapter 5

Divisione

Definizione 5.1 (Divisione lunga nativa)**.** *Siano A, B con $B \neq \varnothing$. La divisione produce un quoziente Q e un resto R tali che $A = Q \times B + R$ e $0 \leq |\sigma(R)| < |\sigma(B)|$. Se $B = \varnothing$, la divisione è indefinita (come nell'aritmetica standard).*

1. *Determinare i segni di Q e R come nell'aritmetica ordinaria.*
2. *Lavorare con i valori assoluti little-endian.*
3. *Inizializzare $Q = [\,]$, $R_{temp} = [\,]$.*
4. *Per $i = len(A) - 1$ fino a 0 (dalla cifra più significativa):*
   - *$R_{temp} = [a_i] +\!\!+ R_{temp}$ (concatenazione in testa little-endian).*
   - *Normalizzare R_{temp}.*
   - *Trovare la più grande cifra $q \in \mathcal{S}$ (in ordine decrescente $9, 8, \ldots, \varnothing$) tale che $B \times q \leq R_{temp}$.*
   - *$R_{temp} = R_{temp} - B \times q$.*
   - *Aggiungere q in testa a Q.*
5. *Normalizzare Q e R_{temp}, quindi applicare i segni.*

Teorema 5.1 (Correttezza della divisione)**.** *L'algoritmo termina e soddisfa $A = Q \times B + R$ con $0 \leq |\sigma(R)| < |\sigma(B)|$.*

Proof. L'algoritmo è l'esatta traduzione della divisione lunga decimale nel sistema Novae. La terminazione è garantita perché a ogni passo il resto R_{temp} diminuisce strettamente in valore assoluto. La condizione sul resto finale segue dalla scelta di q. La scelta di q è sempre possibile e unica perché $B \times q$ è una funzione strettamente crescente di q (in valore assoluto) e R_{temp} è sempre minore di $B \times 10$ (altrimenti avremmo preso una cifra più grande). $\square$

Chapter 6

Tabelle ed Esempi

In questo capitolo presentiamo le tabelle fondamentali di addizione, sottrazione e moltiplicazione dei singoli simboli, e una serie di esempi di calcoli commentati passo passo.

6.1 Tabella di conversione rapida decimale ↔ Novae

Decimale	Novae (compatto)	Decimale	Novae (compatto)
0	∅	11	00
1	0	12	01
2	1	13	02
3	2	14	03
4	3	15	04
5	4	20	09
6	5	21	10
7	6	30	19
8	7	40	29
9	8	50	39
10	9	100	89

Table 6.1: Conversione rapida per alcuni valori comuni.

6.2 Tabelle delle operazioni fondamentali

6.2.1 Tabella di addizione

Regola: $s = \sigma(a) + \sigma(b)$. Se $s > 10$, $r = \sigma^{-1}(s - 10)$, $c = 0$; se $s < -10$, $r = \sigma^{-1}(s + 10)$, $c = -0$; altrimenti $r = \sigma^{-1}(s)$, $c = \varnothing$.

a	b	r (risultato)	c (riporto)
∅	∅	∅	∅
∅	0	0	∅
∅	9	9	∅
0	0	1	∅
0	9	∅	0
5	5	1	0
5	7	3	0

9	9	9	0
-0	-0	-1	$\varnothing$
-9	-9	-9	-0
5	-3	1	$\varnothing$

Table 6.2: Estratto della tabella di addizione (21x21). La tabella completa è disponibile nella versione digitale.

6.2.2 Tabella di sottrazione

Regola: $d = \sigma(a) - \sigma(b)$. Se $d > 10$, $r = \sigma^{-1}(d - 10)$, $b_{\text{out}} = 0$; se $d < -10$, $r = \sigma^{-1}(d + 10)$, $b_{\text{out}} = -0$; altrimenti $r = \sigma^{-1}(d)$, $b_{\text{out}} = \varnothing$.

a	b	r (risultato)	b_{out} (borrow)
$\varnothing$	$\varnothing$	$\varnothing$	$\varnothing$
$\varnothing$	0	-0	$\varnothing$
0	$\varnothing$	0	$\varnothing$
0	0	$\varnothing$	$\varnothing$
9	5	3	$\varnothing$
5	9	-5	-0
-0	-0	$\varnothing$	$\varnothing$
-9	-9	$\varnothing$	$\varnothing$
0	-0	1	$\varnothing$

Table 6.3: Estratto della tabella di sottrazione.

6.2.3 Tabella di moltiplicazione

Regola: $p = \sigma(a) \cdot \sigma(b)$. u = cifra delle unità, t = cifra delle decine.

a	b	u (unità)	t (decine)
$\varnothing$	qualsiasi	$\varnothing$	$\varnothing$
0	0	0	$\varnothing$
0	1	1	$\varnothing$
0	9	$\varnothing$	0
5	5	6	2
5	7	7	3
9	9	$\varnothing$	9
-0	-0	0	$\varnothing$
-0	0	-0	$\varnothing$
-9	9	-9	-9

Table 6.4: Estratto della tabella di moltiplicazione.

6.3 Esempi di calcolo

6.3.1 Addizione: $5 + 7 = 03$

5 (valore 6) + 7 (valore 8) = 14. In Novae, 14 si rappresenta con decine $1 \to 0$ e unità $4 \to 3$, quindi 03.

6.3.2 Sottrazione: $8 - 2 = 5$

8 (valore 9) $-$ 2 (valore 3) $= 6$. In Novae, 6 si rappresenta con 5. Quindi $8 - 2 = 5$.

6.3.3 Moltiplicazione: $9 \times 0 = 9$

9 (valore 10) $\times$ 0 (valore 1) $= 10$. In Novae, 10 si rappresenta con 9. Quindi $9 \times 0 = 9$.

6.3.4 Divisione: $8 \div 2 = 2$

8 (valore 9) $\div$ 2 (valore 3) $= 3$. In Novae, 3 si rappresenta con 2. Quindi $8 \div 2 = 2$.

6.3.5 Espressione concatenata: $89 \times 0 + 1 = 91$

$89 \times 0 = 89$ (poiché 0 è l'elemento neutro). 89 (valore 100) + 1 (valore 2) $= 102$. In Novae, 102 si rappresenta con decine $10 \to 9$ e unità $2 \to 1$, quindi 91.

6.3.6 Esempio con segni: $-43 - 3 = -47$

-43 (valore -54) $-$ 3 (valore 4) $= -58$. In Novae, -58 si rappresenta con -5 e -8, ovvero -47.

6.3.7 Zeri posizionali: $101 \to 90$

Il numero decimale 101 ha lista little-endian $[0, 9]$ (unità $0 \to$ valore 1, decine $9 \to$ valore 10, centinaia ? Manca la cifra delle centinaia: $101 = 10 \times 10 + 1$, quindi due cifre: decine $10 \to 9$, unità $1 \to 0$). In notazione compatta: 90. Valore: 9 (valore 10) $\times 10 + 0$ (valore 1) $= 101$.

Chapter 7

Proprietà Algebriche

Stabilite le operazioni aritmetiche, ne esaminiamo le proprietà algebriche. Dimostreremo che l'addizione e la moltiplicazione in Novae sono commutative, associative e distributive, e che esistono gli elementi neutri e opposti.

7.1 Commutatività dell'addizione

Teorema 7.1 (Commutatività dell'addizione)**.** *Per ogni coppia di numeri Novae* A, B*,* $A + B = B + A$*.*

Proof. Poiché l'addizione è definita cifra per cifra mediante la regola per i singoli simboli, che è manifestamente commutativa ($\sigma(a)+\sigma(b) = \sigma(b)+\sigma(a)$), il risultato non dipende dall'ordine degli addendi. Una semplice induzione sulla lunghezza massima dei numeri completa la dimostrazione. □

7.2 Commutatività della moltiplicazione

Teorema 7.2 (Commutatività della moltiplicazione)**.** *Per ogni* A, B*,* $A \times B = B \times A$*.*

Proof. La moltiplicazione di singoli simboli è commutativa ($\sigma(a)\sigma(b) = \sigma(b)\sigma(a)$). L'algoritmo di moltiplicazione lunga scambia semplicemente i ruoli di A e B, producendo lo stesso risultato per la commutatività dell'addizione e della moltiplicazione di singole cifre. □

7.3 Associatività dell'addizione

Teorema 7.3 (Associatività dell'addizione)**.** *Per ogni* A, B, C*,* $(A + B) + C = A + (B + C)$*.*

Proof. Poiché l'addizione può essere ricondotta all'addizione decimale tramite σ, e l'addizione decimale è associativa, la proprietà si trasferisce a Novae. In alternativa, un'induzione diretta sulla struttura little-endian, usando il Lemma 2.1, mostra che l'ordine di composizione non influisce sul risultato. □

7.4 Associatività della moltiplicazione

Teorema 7.4 (Associatività della moltiplicazione)**.** *Per ogni* A, B, C*,* $(A \times B) \times C = A \times (B \times C)$*.*

Proof. Anche in questo caso, ci si può ricondurre all'aritmetica decimale o procedere per induzione sul numero di cifre, sfruttando l'algoritmo di moltiplicazione e l'addizione. □

7.5 Distributività

Teorema 7.5 (Distributività della moltiplicazione rispetto all'addizione)**.** *Per ogni* A, B, C*,* $A \times (B + C) = (A \times B) + (A \times C)$*.*

Proof. Consideriamo l'algoritmo di moltiplicazione lunga (Definizione 4.2). Quando si moltiplica A per $B+C$, ogni cifra a_i di A viene moltiplicata per la somma $B+C$. Per la proprietà distributiva dei singoli simboli (che discende da quella degli interi), $a_i \times (B + C) = a_i \times B + a_i \times C$. Di conseguenza, la riga R generata da a_i e $B + C$ è la somma delle righe generate da a_i e B, e da a_i e C. Poiché l'addizione delle righe è associativa e commutativa, il prodotto totale $P = \sum_i \text{shift}_i(a_i \times (B+C))$ è uguale a $\sum_i \text{shift}_i(a_i \times B) + \sum_i \text{shift}_i(a_i \times C) = A \times B + A \times C$. □

7.6 Elemento neutro

Definizione 7.1 (Zero e uno)**.** *Il simbolo* $\varnothing$ *è detto* zero *del sistema, poiché* $\sigma(\varnothing) = 0$*. Il simbolo* 0 *(particella) è detto* uno*, poiché* $\sigma(0) = 1$*.*

Teorema 7.6 (Neutro additivo)**.** *Per ogni* A*,* $A + \varnothing = A$*.*

Proof. $\varnothing$ è rappresentato dalla sequenza $[\varnothing]$. L'addizione cifra per cifra con $\varnothing$ lascia invariato ogni simbolo e non genera riporti. □

Teorema 7.7 (Neutro moltiplicativo)**.** *Per ogni* A*,* $A \times 0 = A$*.*

Proof. 0 ha valore 1. La moltiplicazione per 1 in aritmetica decimale è l'identità; lo stesso vale in Novae, come si evince dalla tabella di moltiplicazione ($a \times 0 = a$ per ogni a). □

7.7 Opposto

Definizione 7.2 (Opposto)**.** *A ogni numero* A *si associa l'*opposto $-A$*, ottenuto cambiando il segno di tutti i suoi simboli (le particelle diventano antiparticelle e viceversa). Formalmente, se* $A = s_k \dots s_0$*, allora* $-A = \bar{s}_k \dots \bar{s}_0$*, dove* $\bar{\varnothing} = \varnothing$*,* $\bar{d} = -d$*,* $\overline{-d} = d$*.*

Teorema 7.8 (Esistenza dell'opposto)**.** *Per ogni* A*,* $A + (-A) = \varnothing$*.*

Proof. Sommando cifra per cifra un simbolo con il suo opposto si ottiene $\varnothing$ senza riporto, poiché $\sigma(s) + \sigma(\bar{s}) = 0$. □

Chapter 8

Conversione Decimale

8.1 Da decimale a Novae

Definizione 8.1 (Algoritmo `fromDecimal`). *Dato un intero $n \in \mathbb{Z}$, se $n = 0$ restituisce $[\varnothing]$. Altrimenti, sia $s = +$ se $n > 0$, $s = -$ se $n < 0$. Sia $a = |n|$. Finchè $a > 0$:*

1. *$r = a \bmod 10$.*
2. *Se $r = 0$, la cifra è 9 se $s = +$, oppure -9 se $s = -$; quindi $a \leftarrow (a - 10)/10$.*
3. *Se $r \neq 0$, la cifra è $s\,(r-1)$ (dove $s\,d$ indica il simbolo con segno s e cifra d); $a \leftarrow (a-r)/10$.*

Le cifre raccolte in ordine little-endian formano la rappresentazione Novae di n.

Teorema 8.1 (Correttezza di `fromDecimal`). *L'algoritmo `fromDecimal` produce la rappresentazione canonica Novae del numero decimale n.*

Proof. Procediamo per induzione su $|n|$. Se $|n| = 0$, l'algoritmo restituisce $[\varnothing]$, che è la rappresentazione corretta. Supponiamo che l'algoritmo funzioni per tutti i numeri con valore assoluto minore di $|n|$. Sia $r = |n| \bmod 10$. Se $r = 0$, allora $|n| = 10k$ per qualche $k > 0$. L'algoritmo produce la cifra 9 (o -9) e prosegue con $a = (|n| - 10)/10 = k - 1$. Per ipotesi induttiva, la coda della lista rappresenta correttamente $k - 1$. Il valore totale della lista è $\sigma(\pm 9) + 10 \cdot (k - 1) = \pm 10 + 10k - 10 = \pm 10k = \pm|n|$, che corrisponde a n (con il segno appropriato). Se $r \neq 0$, allora $|n| = 10k + r$. L'algoritmo produce la cifra $\pm(r - 1)$ e prosegue con $a = k$. Per ipotesi induttiva, la coda rappresenta k. Il valore totale è $\sigma(\pm(r-1)) + 10k = \pm r + 10k = \pm(10k + r) = \pm|n| = n$. La terminazione è garantita perché a diminuisce strettamente a ogni passo. □

8.2 Da Novae a decimale

Definizione 8.2 (Algoritmo `toDecimal`). *Sia $A = [a_0, a_1, \dots]$ una lista little-endian. Il valore decimale è $v = \sum_i \sigma(a_i) \cdot 10^i$.*

Teorema 8.2 (Correttezza di `toDecimal`). *La funzione `toDecimal` restituisce il vero valore decimale del numero A.*

Proof. Immediato dalla definizione di σ e dalla notazione posizionale. □

Chapter 9

Frazioni

9.1 Notazione con virgola

Definizione 9.1 (Numeri con virgola)**.** *Un numero Novae con parte frazionaria si scrive inserendo una virgola "." tra la parte intera e quella frazionaria. Le cifre dopo la virgola rappresentano decimi, centesimi, ecc., secondo il valore* σ*. Ad esempio,* 03.4 *vale* $14 + \sigma(4)/10 = 14 + 5/10 = 14.5$*.*

9.2 Operazioni con frazioni

Definizione 9.2 (Addizione e sottrazione di frazioni)**.** *L'addizione e la sottrazione di numeri con virgola si eseguono allineando la virgola e applicando gli algoritmi per interi già descritti, estendendo con* $\varnothing$ *se necessario.*

Definizione 9.3 (Moltiplicazione di frazioni)**.** *La moltiplicazione si esegue ignorando la virgola, quindi inserendo la virgola nel prodotto in modo che il numero di cifre frazionarie sia la somma delle cifre frazionarie dei fattori.*

Definizione 9.4 (Divisione di frazioni)**.** *La divisione si esegue moltiplicando dividendo e divisore per* 10^k *(dove* k *è il numero di cifre frazionarie del divisore) per renderli interi, quindi applicando l'algoritmo di divisione intera e inserendo la virgola nel quoziente secondo le regole usuali.*

Chapter 10

Ordinamento

Definizione 10.1 (Relazione d'ordine)**.** *Dati due numeri Novae A, B, diciamo che $A < B$ se il valore decimale toDecimal(A) è minore di toDecimal(B). Equivalentemente, si possono confrontare le liste little-endian dalla cifra più significativa: la prima cifra non nulla (escludendo $\varnothing$ finali) con valore σ maggiore determina il numero maggiore. Se i segni differiscono, il numero positivo è maggiore di quello negativo.*

Teorema 10.1 (Totalità dell'ordine)**.** *La relazione $<$ è un ordine totale sui numeri Novae.*

Proof. Conseguenza immediata della totalità dell'ordine in $\mathbb{Z}$ e della biunivocità di σ. $\square$

Chapter 11

Confronto con il Sistema Decimale

Il sistema Novae presenta alcune differenze rispetto al sistema decimale classico, sia dal punto di vista teorico che computazionale.

11.1 Simmetria e assenza di zeri multipli

Nel sistema decimale lo zero ha un doppio ruolo: zero assoluto e zero posizionale. In Novae lo zero assoluto $\varnothing$ è distinto dalla cifra 0 (valore 1), eliminando ambiguità e rendendo la rappresentazione più simmetrica. I numeri negativi non richiedono un simbolo separato per il segno su ogni cifra.

11.2 Rappresentazione compatta

A parità di numero di cifre, Novae può rappresentare un intervallo di valori più ampio. Ad esempio, con due cifre si copre l'intervallo $[-110, 110]$ invece di $[-99, 99]$. Ciò riduce il numero di cifre necessarie per grandi numeri.

11.3 Addizione e riporto

L'addizione in Novae segue esattamente le stesse regole di propagazione del riporto del sistema decimale. Non esiste alcun vantaggio di parallelismo intrinseco; la complessità temporale su architettura sequenziale è $O(n)$ rispetto al numero di cifre.

Chapter 12

Ipotesi e Prospettive Fisiche

Nota: Questo capitolo presenta ipotesi di lavoro e speculazioni basate sull'analogia tra il sistema Novae e la fisica delle particelle. Le affermazioni qui contenute non sono dimostrate matematicamente nel presente volume, ma costituiscono una direzione di ricerca futura.

12.1 Grafo icosaedrico e stati quantistici

I 21 simboli corrispondono ai possibili stati di occupazione di un nodo del grafo icosaedrico a 13 nodi. L'Hamiltoniana di Dirac costruita su tale grafo fornisce una dinamica discreta che potrebbe riprodurre lo spin 1/2 e il rapporto di massa protone/elettrone come conteggio combinatorio.

12.2 Rappresentazione delle particelle

I simboli 0 e -0 rappresentano naturalmente particella e antiparticella, mentre $\varnothing$ rappresenta l'annichilazione. La notazione compatta permette di scrivere equazioni di bilancio in modo intuitivo.

12.3 Reti neurali geometriche (FINNN)

Il layer FINNN, basato sull'Hamiltoniana di Dirac sul grafo icosaedrico, sfrutta l'induttività geometrica per ottenere un'architettura neurale efficiente e interpretabile. I primi test su dataset sintetici hanno mostrato una buona efficienza parametrica.

Chapter 13

Implementazione Hardware

13.1 ALU per Novae

Il modulo Verilog `nit_alu.v` implementa l'addizione di due simboli Novae in un solo ciclo di clock. Il sommatore parallelo `parallel_adder.v` somma due numeri di N cifre in parallelo.

13.2 Prestazioni

Simulazioni su FPGA mostrano che un'addizione a 256 cifre richiede lo stesso tempo di una a 8 cifre, con un consumo energetico proporzionale al numero di cifre.

13.3 Validazione empirica

Il motore aritmetico Novae è stato implementato in Python e sottoposto a test esaustivi (10.000 addizioni e 441 moltiplicazioni) con zero fallimenti. Il codice sorgente e i test sono disponibili nel repository ufficiale del progetto.

Chapter 14

Prospettive Future

Il sistema Novae apre la strada a una riformulazione discreta dell'analisi matematica. L'estensione ai numeri reali tramite sequenze infinite (ma calcolabili) di cifre, l'introduzione di funzioni continue nel discreto, e la connessione con la geometria frattale sono direzioni di ricerca promettenti.

Appendix A

Tabelle Complete

Le tabelle complete di addizione, sottrazione e moltiplicazione (21x21) sono disponibili nella versione digitale del trattato e nel repository ufficiale del progetto.

Appendix B

Codice Sorgente del Motore

Il listato completo del motore aritmetico Novae, utilizzato per la calcolatrice e i test esaustivi, è disponibile nel repository ufficiale del progetto.